



임베디드
엔지니어,

안준영



학교 / 전공

아주대학교 전자공학과 학사
(2020.03 ~ 2026.02)



성명

안준영 (2020.12.15)



연락처

010-5717-1215



E-mail

ajy4452@ajou.ac.kr



GitHub

github.com/ajelec20ajy

프로젝트 파일 및 보고서/PPT

- ## [1] 목표 및 담당

- (1) 실시간 위치 추정
- (2) 조향 / 경로 계산
- (3) 직진 구동 모터
- (4) LCD 주행 상태 표시

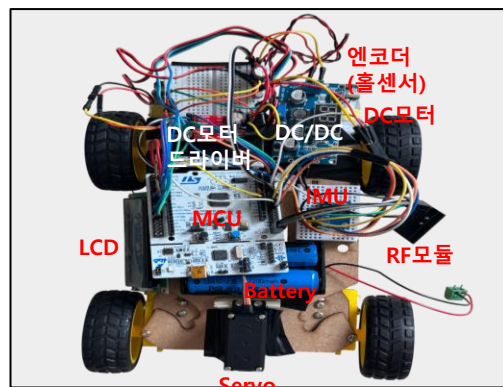
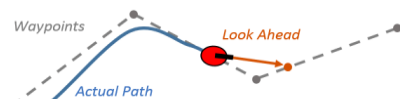
[2] 진행 내용

- (1) 구성 기준
- (2) 우선 순위 / 실행 빈도
- (3) 데이터 공유 설계

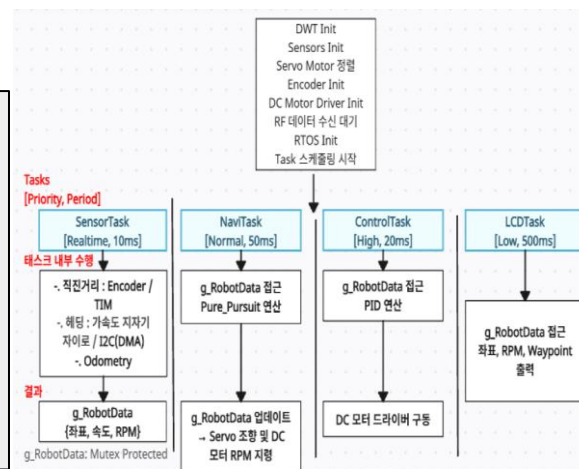
- (1) 하드웨어 제어 / Application 계층 분리
- (2) DC 모터 FF+PI 제어 튜닝 HILS
- (3) 주행 구간별 DC 모터 가변 RPM 제어
- (4) 능동 파라미터 기반 Pure Pursuit
- (5) 실시간성 최적화 (I2C 통신 개선)

[3] 결과 및 배운점

- ### (1) 목표 제어 시스템



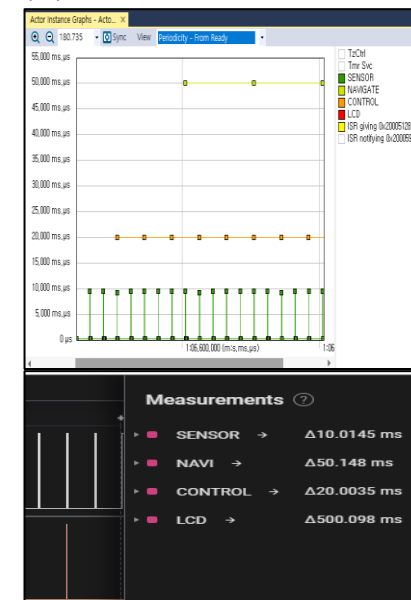
(2) RTOS 구성



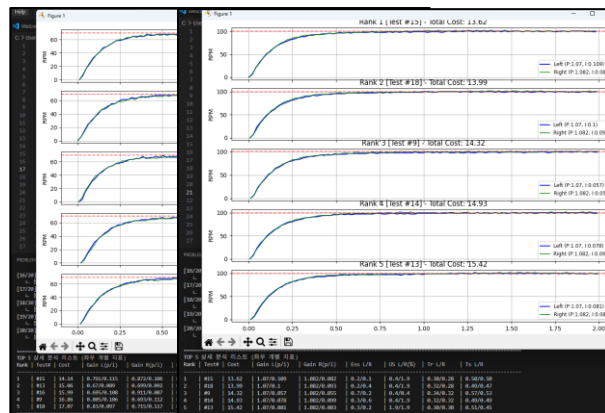
(3) SW 아키텍처



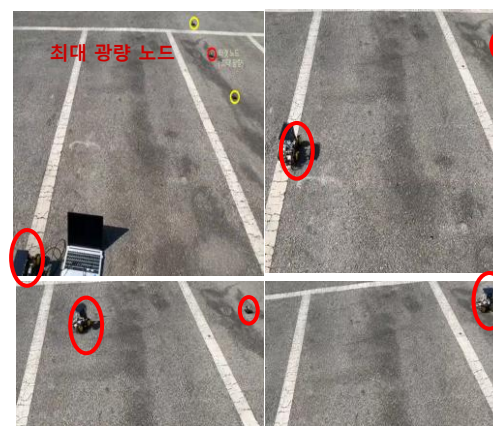
(4) 태스크 주기



(4) FF+PI 제어 튜닝 HILS



(5) 주행 정확도



자 측정	X[m]	Y[m]	오차[cm]	오차[%]	3cm오차[%]
1회차	2.976	2.984	2.88	0.679	11.56
2회차	2.977	2.982	2.92	0.688	7.94
3회차	2.975	2.984	2.97	0.7	3.18
4회차	2.979	2.982	2.77	0.653	23.41
5회차	2.984	2.98	2.56	0.603	43.88
6회차	2.977	2.981	2.98	0.702	1.67
7회차	2.98	2.983	2.62	0.618	37.51
8회차	2.978	2.983	2.78	0.655	21.97
9회차	2.981	2.978	2.91	0.686	9.31
10회차	2.982	2.98	2.69	0.634	30.93
평균	2.9789	2.9817	2.81	0.662	19.14

프로젝트 2 : 임베디드시스템실험 기능 분산을 위한 제어기 결합 구조 설계

- 다중 MCU 연결을 위한 안정적인 3단 계층 구조 시스템 구현
- STM32F103C8T6, 라즈베리파이 제로2W / STM32CubeIDE, SSH 접속

[1] 계기 및 목표

- MCU간 연동
FSM / 상태 결정 권한 중앙화
- 중앙집중형 구조 모사

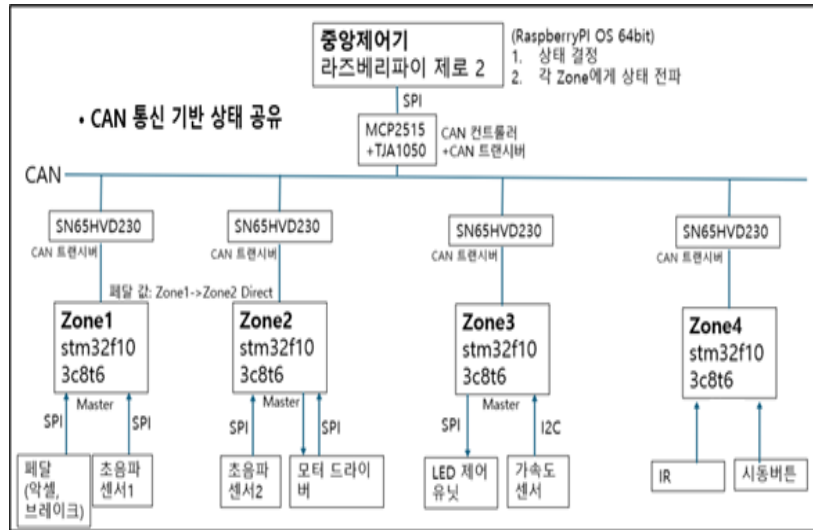
[2] 구현 내용

- 3단 계층 구조 설계
- (1) 중앙: 시스템 통제 권한
- (2) 중간: 로컬 통신

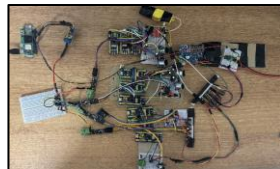
- CAN 통신 설계

- (1) 비상 메시지 우선 ID
- (2) 부하량 및 주기 검사

(1) 3단 계층 구조



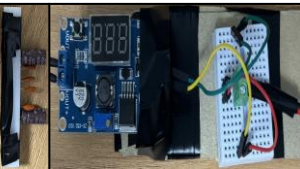
• 전체 사진



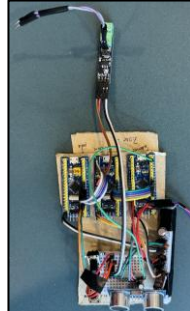
• 중앙 제어기



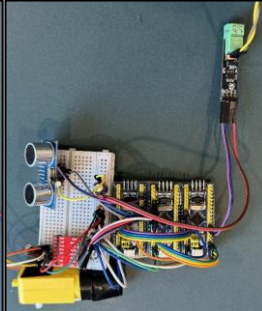
• 전원



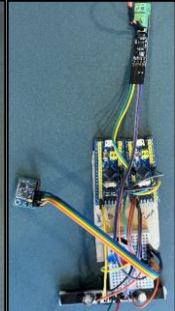
• Zone1



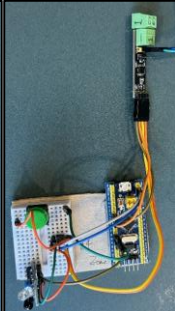
• Zone 2



• Zone 3



• Zone 4



(2) CAN 메시지 ID

#define ID_1_SONIC 0x010 // 초음파1 메시지	#define ID_3_ACC 0x012 // 가속도 메시지
#define ID_1_PEDAL 0x040 // 페달 데이터 메시지	#define ID_3_HB 0x073 // Zone3 HB
#define ID_1_HB 0x071 // Zone1 HB	#define ID_4_IR 0x142 // IR센서 메시지
// Zone1 고장 알림 메시지	#define ID_4_BTN 0x144 // 시동버튼 메시지
#define ID_1_PEDAL_SPI_ERR 0x022	#define ID_4_HB 0x074 // Zone4 HB
#define ID_2_SONIC 0x011 // 초음파2 메시지	#define ID_CENTRAL_MOTOR_ERR 0x001
#define ID_2_MOTOR_STATUS 0x110 // 모터 RPM	#define ID_CENTRAL_EMERGENCY 0x002
#define ID_2_HB 0x072 // Zone2 HB	#define ID_CENTRAL_STATUS 0x004
// Zone2 고장 알림 메시지	
#define ID_2_MOTOR_SPI_ERR 0x020	

(3) CAN 버스 부하 예측 및 데이터 구조

노드	주기가 있는 데이터 전송량
중앙 제어기	Hearbeat 100Hz
Zone1	초음파 10Hz, 페달 10Hz, Hearbeat 100Hz
Zone2	초음파 10Hz, 모터 드라이버 PWM 100Hz, Hearbeat 100Hz
Zone3	가속도 100Hz, Hearbeat 100Hz
Zone4	Hearbeat 100Hz
부하 예측	
8 바이트 데이터 기준 메시지 전송 주기 1Hz 당 125bits/s (프레임 단위)	
주기적 데이터 총량 = 730msg/s = 91250 bits/s	
91250 [bits/s] / 500000bps = 18.25%	

```
typedef union {
    struct {
        uint16_t data;
        uint8_t reserved[4];
        uint8_t counter;
        uint8_t checksum;
    } __attribute__((packed)) data16;
    struct {
        uint16_t data1;
        uint16_t data2;
        uint8_t reserved[2];
        uint8_t counter;
        uint8_t checksum;
    } __attribute__((packed)) data32;
    uint8_t bytes[8];
} CAN_Payload_t; // 데이터 내부 패킷
```

(4) CAN 버스 부하 및 메시지 주기

ajy@ajy:~\$ canbusload can0@500000	
can0@500000	796 107460 50944 0 21%
can0@500000	804 108540 51456 0 21%
can0@500000	804 108540 51456 0 21%
can0@500000	804 108540 51456 0 21%
can0@500000	802 108270 51328 0 21%
can0@500000	799 107865 51136 0 21%

99 ms	ID	data ...	<
00010	004	04 00 00 00 00	
00099	010	1E 00 00 00 00	
00099	011	BB 00 00 00 00	
00010	012	00 00 00 00 65	
00009	040	B8 00 00 00 00	
00010	071	04 00 00 00 00	
00009	072	04 00 00 00 00	
00010	073	04 00 00 00 00	
00011	074	04 00 00 00 00	

프로젝트 2 : 임베디드시스템실험 기능 분산을 위한 제어기 결합 구조 설계

[1] 목표

- **고장 상황 통제**
대응 동작 / 고장 전파 방지

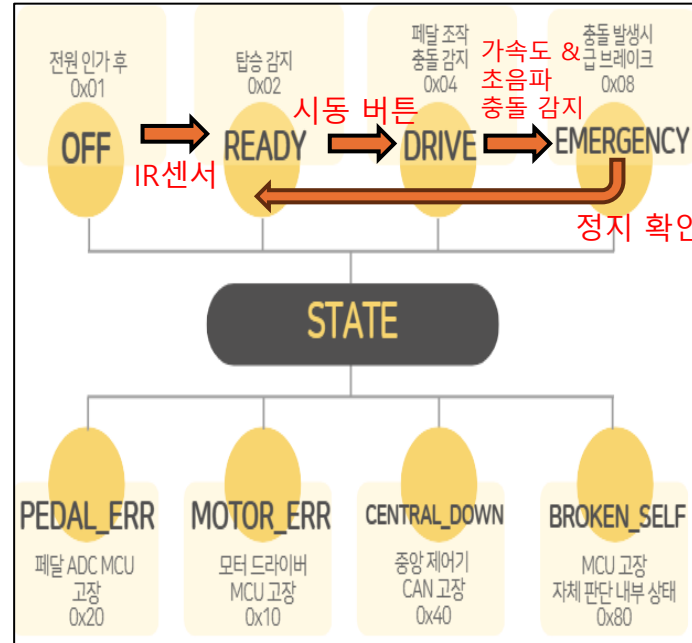
[2] 진행 내용

- **Fault 파악 및 Failure 방지**
 - (1) 고장 요소 파악
 - (2) 에러 감지
 - (3) 안전 상태 천이

[3] 배운점

- MCU SW 계층적 설계 및 구현 역량
- 시스템 이해 및 설계 역량

(1) FSM



(2) 고장 분류

		에러 주체				
		중앙 제어기	Zone1	Zone2	Zone3	Zone4
에러 원인	CAN Heartbeat	CENTRAL_DOWN	(1) BROKEN_SELF (2) EMERGENCY	(1) BROKEN_SELF (2) EMERGENCY	ISOLATION	ISOLATION
	CAN Corruption	CENTRAL_DOWN	EMERGENCY	EMERGENCY	ISOLATION	ISOLATION
	LOCAL SPI		(1) PEDAL_ERR (2) ISOLATION	(1) MOTOR_ERR (2) ISOLATION	ISOLATION	

(3) 고장 대응 예시

1) CENTRAL_DOWN

```

[13418751] 시스템 상태 요약 = can0 000 [8] 6A
현재 모드: DRIVE can0 000 [8] 60
초음파 거리: Z1: 16 cm can0 000 [8] 60
가속도: 1.620000 can0 000 [8] 6A
ID: 0x040 can0 000 [8] 6A
페달 데이터 can0 000 [8] 6A
중요 정보 can0 000 [8] 6F
can0 000 [8] 71
can0 000 [8] 71
can0 000 [8] 6F
  
```

모터의 페달 조작 이외의 모든 기능 정지

2) PEDAL_ERR

```

Zone2-페달 SPI
패킷 고의 에러

tx_buf.pkt.header = 0xAA;
tx_buf.pkt.data = pedal;
tx_buf.pkt.status = 0;
tx_buf.pkt.seq = seq++;
tx_buf.pkt.seq = 0; // PEDAL_ERR
tx_buf.pkt.reserved = 0x00;
tx_buf.pkt.crc = crc_xor(pedal, tx_buf.pkt.reserved);
tx_buf.pkt.tail = 0xED;
  
```

Zone2에서 PEDAL_ERR 상태시 모터 정지

프로젝트 3 : IoT 프로젝트 불안정한 WiFi 환경을 보완한 상태 감시 IoT 시스템

- WiFi 단절과 브로커 다운 상황에서도 데이터 유실 없이 동작하는 상태 감시 노드 개발
- ESP32 WROOM 개발 보드 / Arduino IDE

[1] 목표 및 담당

- 통신 실패 상황 대응

(1) 브로커 이중화 / 데이터 보존

- 제한된 자원 운용

(1) 배터리 / 무선 통신 자원

[2] 진행 내용

- FreeRTOS 설계

(1) 듀얼 코어 역할 분리

(2) 우선순위 / 실행 주기

- Failover & Local 링버퍼

(1) 백업 브로커 전환

(2) 로컬 저장 후 재발행

- 발행 주기 제어

[3] 결과 및 배운점

- 브로커 전환 및 데이터 재발행

- 상태 기반 가변 발행 주기

- RTOS 설계 역량

- 자료구조(링버퍼) 활용 역량

- 고장 대응 설계 역량

- 임베디드 시스템 구조 이해

(1) MQTT 발행 및 UI 구성

```
c:\Program Files\mosquitto>mosquitto_sub -h broker.emqx.io -p 1883 -t "sensor/ajy"
{"temp":23.80,"hum":80.00,"sound":753,"interval":1362,"period":1300}
{"temp":23.80,"hum":80.00,"sound":753,"interval":1362,"period":1300}
{"temp":nan,"hum":80.00,"sound":753,"interval":2674,"period":1300}
{"temp":23.80,"hum":80.00,"sound":752,"interval":1362,"period":1300}
{"temp":23.70,"hum":75.00,"sound":755,"interval":1362,"period":1300}
{"temp":23.70,"hum":75.00,"sound":755,"interval":1362,"period":1300}
{"temp":23.70,"hum":80.00,"sound":762,"interval":1361,"period":1300}
{"temp":23.70,"hum":80.00,"sound":754,"interval":1311,"period":1300}
{"temp":23.70,"hum":80.00,"sound":743,"interval":1311,"period":1300}
```

메인 서버
메인 토픽

```
C:\Program Files\mosquitto>mosquitto_sub -h test.mosquitto.org -p 1883 -t "sensor/ajy"
{"temp":23.80,"hum":81.00,"sound":756,"interval":2281,"period":2500}
{"temp":23.80,"hum":80.00,"sound":757,"interval":2502,"period":2500}
{"temp":23.80,"hum":80.00,"sound":755,"interval":2502,"period":2200}
{"temp":23.80,"hum":81.00,"sound":752,"interval":2282,"period":2200}
{"temp":23.80,"hum":80.00,"sound":754,"interval":2282,"period":2200}
{"temp":23.80,"hum":80.00,"sound":754,"interval":2282,"period":2200}
```

백업 서버
메인 토픽

```
C:\Users\ajy97>cd "C:\Program Files\mosquitto"
C:\Program Files\mosquitto>mosquitto_sub -h test.mosquitto.org -p 1883 -t "sensor/events"
{"temp":23.80,"hum":81.00,"sound":899,"time":"2022-12-01 18:07:32"}
{"temp":23.80,"hum":80.00,"sound":890,"time":"2022-12-01 18:07:32"}
{"temp":23.70,"hum":80.00,"sound":893,"time":"2022-12-01 18:07:33"}
```

백업 서버
이벤트 토픽



(2) 상태 기반 발행 주기 제어

```
----- [PeriodManager Snapshot] -----
RAW => RSSI: -43 Battery: 349 Wi-Fi OK: YES MQTT OK: YES
PERIOD=> RSSI: 1500 Battery: 3000 Failover: 1000
FINAL => 1500 ms (onBackup=NO)

===== TEST: Simulating PRIMARY MQTT FAILURE after 10s =====
>>> PRIMARY MQTT set to failure mode (port 81)
[Failover] PRIMARY down -> Try reconnect PRIMARY...
[Failover] PRIMARY DEAD -> Switching to BACKUP
Attempting MQTT connection... [MQTT] LOST connection
[PublishTask] MQTT DEAD -> skipped
----- [PeriodManager Snapshot] -----
RAW => RSSI: -43 Battery: 349 Wi-Fi OK: YES MQTT OK: NO
PERIOD=> RSSI: 1500 Battery: 3000 Failover: 3000
FINAL => 2700 ms (onBackup=YES)
```

백업 서버 사용 여부 기반 발행 주기 제어

```
----- [PeriodManager Snapshot] -----
RAW => RSSI: -20 Battery: 4095 Wi-Fi OK: YES MQTT OK: YES
PERIOD=> RSSI: 1000 Battery: 1000 Failover: 1000
FINAL => 1000 ms (onBackup=NO)

----- [PeriodManager Snapshot] -----
RAW => RSSI: -41 Battery: 4095 Wi-Fi OK: YES MQTT OK: YES
PERIOD=> RSSI: 1500 Battery: 1000 Failover: 1000
FINAL => 1100 ms (onBackup=NO)

----- [PeriodManager Snapshot] -----
RAW => RSSI: -70 Battery: 4095 Wi-Fi OK: YES MQTT OK: YES
PERIOD=> RSSI: 3000 Battery: 1000 Failover: 1000
FINAL => 1400 ms (onBackup=NO)
```

WiFi RSSI 기반 발행 주기 제어

(3) RTOS 태스크 구성

코어	태스크	우선순위/기능	주기 [ms]
Core 0	Publish	6/ MQTT 발행	가변
	Failover	7/ 백업 서버 전환	200
	EventUpload	2/ 이벤트 발행	500
Core 1	Sensor	3/ 센서데이터 취득	500
	RSSI	4/ RSSI 측정	3000
	Battery	4/ 배터리 측정	3000
	PeriodManager	5/ 전송 주기 계산	50